

Это журнал всех проделанных действий в ходе разработки и комментариев к ним. Этапы работы разделены на блоки и имеют цели, ожидаемый результат и полученный результат.

1. Зафиксировать текущее состояние проекта

Цель: зафиксировать текущее состояние проекта после всех переносов и экспериментов.

Что нужно сделать:

- собрать текущий проект в ESP-IDF (Windows);
- проверить, что BLE-часть запускается;
- понять, какие модули уже есть в коде;
- зафиксировать, что работает, а что нет.

Готово, если:

- проект собирается без критических ошибок;
- BLE-инициализация проходит;
- есть список проблем / ограничений;
- есть запись в рабочем дневнике.

Сделано:

- Ethernet поднялся и получает IP.
- TCP подключение настроено.
- Добавил таймауты для TCP (пока что убрал для отладки).
- Настроил транспортный уровень TCP.
- Настроил архитектуру проекта.
- Создал собственные события.
- Реализовал очередь для событий.
- BLE часть запускается, но происходит brownout при просадке напряжения. Временное решение - отключить Hardware brownout detector в sdkconfig. Постоянное решение - нормальный блок питания или конденсатор между 3V3 и gnd на 300-400 uf.

2. Минимальный BLE-сканер в ESP-IDF

Цель: реализовать минимально работающий BLE-сканер.

Что нужно сделать:

- изучить документацию ESP-IDF по BLE scanning;
- запустить continuous scan;
- выводить MAC-адрес и RSSI обнаруженных устройств;
- проверить стабильность и частоту сканирования.

Готово, если:

- BLE-сканер работает;
- данные выводятся в лог;
- понятны ограничения ESP32 по BLE.

Сделано:

- Непрерывное сканирование запущено.

- Выводится RSSI и MAC-адрес обнаруженных устройств.
- Устройство стабильно, при условии нормального питания. Частота сканирование 50 мс.

3. Формирование структуры BLE-события

Цель: описать и внедрить структуру BLE-события.

Что нужно сделать:

- описать структуру `ble_event_t`;
- включить поля: MAC, RSSI, `adv_data`, `timestamp`;
- привязать формирование события к BLE callback.

Готово, если:

- события формируются как структуры;
- код компилируется;
- логика понятна и расширяема.

Сделано:

- Описана структура `ble_event_t`.
- Включены поля: MAC, RSSI, `adv_data`, `timestamp`.
- Привязано формирование события к BLE callback.

4. Реализация монотонного таймера

Цель: добавить монотонное время к BLE-событиям.

Что нужно сделать:

- реализовать `monotonic timer` через `esp_timer`;
- добавить поле `monotonic_ms` в `ble_event_t`;
- проверить стабильность таймера.

Готово, если:

- каждое событие имеет корректный `monotonic timestamp`;
- нет скачков и переполнений;
- код стабилен.

Сделано:

- Реализован `monotonic timer` через `esp_timer`.
- Добавил поле `monotonic_ms` в `ble_event_t`.
- События имеют корректный `timestamp`.
- Скачков и переполнений нет.

5. Архитектура синхронизации времени (SNTP)

Цель: продумать архитектуру синхронизации времени.

Что нужно сделать:

- изучить SNTP в ESP-IDF;
- определить, где хранить `offset` времени;
- продумать флаг `time_synced`;
- реализовать заглушку логики (без реального NTP).

Готово, если:

- понятна схема синхронизации времени;
- есть TODO-лист по доработке;
- код подготовлен к интеграции SNTP.

Сделано:

- Флаг `time_synced` есть.
- Добавлен DHCP. ESP получает адрес назначенный системой 192.168.137.X, компьютер получает адрес 192.168.137.1.
- SNTP есть. Выход в интернет есть, для этого нужно зайти в Панель управления -> Сеть и интернет -> Центр управления сетями и общим доступом -> Изменение параметров адаптера -> Правой кнопкой мыши по Беспроводная сеть Wi-Fi сети -> Свойства -> Доступ и установить галочку "Разрешить другим пользователям сети использовать подключение к Интернету данного компьютера, в "Подключение домашней сети" выбрать Ethernet подключенного устройства.
- Время между обнаружением BLE устройства и отправки его на сервер = 2 мс.
- Точность абсолютной временной метки utc составляет ~60 мс. Реальный NTP - SNTP ESP. То есть в момент получения сервером пакета с данными от esp текущее время utc на 60 мс больше чем вычисленное utc на esp.

6. Проектирование Protobuf-схемы

Цель: подготовить Protobuf-схему для передачи данных.

Что нужно сделать:

- спроектировать сообщения `BleEvent`, `DeviceInfo`, `ScanBatch`;
- определить типы и поля;
- оформить `.proto` файл;
- добавить `packet_proto_builder`;
- добавить буферизацию для protobuf;
- описать схему в текстовом виде для согласования.

Готово, если:

- есть готовый `.proto`;
- структура данных понятна;
- документ готов к обсуждению.

Сделано:

- Установка компилятора `proto`.

Перейти на <https://github.com/protocolbuffers/protobuf/releases> и скачать архив `protoc-33.6-win64.zip`. Распаковать его в `C:\Program Files\protobuf`. Должно получиться "`C:\Program Files\protobuf\bin\protoc.exe`".

Далее добавить путь "`C:\Program Files\Protobuf\bin`" в "Переменные среды" в "Path".

- Установка библиотеки для ESP.

Для esp используется облегченная версия `protobuf Nanopb`. Переходим на <https://github.com/nanopb/nanopb> и скачиваем архив. В корне проекта создаем папку `components` и в нее распаковываем архив (не должно быть вложенных папок). Получается `wt32_eth01_ble_eth\components\nanopb`.

Далее удаляем старый `CMakeLists.txt` и создаем новый с содержимым:

```
idf_component_register(  
    SRCS  
        pb_common.c  
        pb_encode.c  
        pb_decode.c  
    INCLUDE_DIRS  
        .  
)
```

Теперь в этой папке можно удалить все кроме:

`pb.h`,
`pb_common.c`,

```
pb_common.h,  
pb_decode.c,  
pb_decode.h,  
pb_encode.c,  
pb_encode.h,  
CMakeLists.txt.
```

— Создаем proto файл.

В корне проекта создаем папку proto и в ней файл scan.proto. Получается wt32_eth01_ble_eth\proto\scan.proto. В этот файл добавляем содержимое:

```
syntax = "proto3";  
package ripas;  
message BleEvent {  
  bytes mac = 1;  
  sint32 rssi = 2;  
  bytes adv_data = 3;  
  uint64 utc_ms = 4;  
  uint64 monotonic_ms = 5;  
  bool time_sync = 6;  
}  
message DeviceInfo {  
  string device_id = 1;  
  string fw_version = 2;  
}  
message ScanBatch {  
  DeviceInfo device = 1;  
  repeated BleEvent ble = 2;  
  uint64 batch_seq = 3;  
}
```

В этой же папке создаем файл scan.options. Получается

wt32_eth01_ble_eth\proto\scan.options. В этот файл добавляем содержимое:

```
ripas.BleEvent.mac max_size:6  
ripas.BleEvent.adv_data max_size:31
```

— Генерация C-кода из .proto.

Копируем из архива папку generator в каталог проекта в soft\nanorpb. Добавить папку generator в в "Переменные среды" в "Path". Проверяем: в терминале вводим "protoc --nanorpb_out=.", если нет ошибок то все ок.

Устанавливаем Python зависимости. В корне проекта выполняем "pip install protobuf grpcio-tools" и проверяем "python -c 'import google.protobuf'", если ошибок нет то все ок. Чтобы сгенерировать proto/scan.pb.c и proto/scan.pb.h, выполняем в корне проекта "protoc --nanorpb_out=. proto/scan.proto". В папке proto появляются два файла. Переносим их в "wt32_eth01_ble_eth\main". Добавляем scan.pb.c в CMakeLists проекта. В scan.pb.c указываем корректный путь до scan.pb.h (#include "scan.pb.h").

— Переход от packet_builder к packet_proto_builder.

Файл protobuf_builder описывает логику инициализации сообщения, заполнения полей и сериализацию данных.

По умолчанию тип bytes (в .proto) при генерации в с++ превращается в структуру pb_callback_t с динамическим (переменным) выделением памяти, чтобы сделать статическое (постоянное) выделение памяти создаем файл scan.options (описывалось выше). Теперь тип bytes при генерации превращается в структуру pb_bytes_array_t с

полями `size` (размер) и `bytes` (данные). При заполнении соответствующего поля в сообщении (например `mac`), обращаться напрямую к этим полям (`mac.size` и `mac.bytes`).
— Буферизация данных `protobuf`.

Проанализировать размер `proto`-пакетов и при необходимости сделать `buffer[256]` для хранения батчей в режиме офлайн. Если размер `proto`-пакетов ≤ 64 , то оставить буферизацию как есть.

Максимальный наблюдаемый размер `proto`-пакетов 56 байт. Значит оставляем буферизацию как есть.

— Декодирование на сервере.

Устанавливаем библиотеку `protobuf`. В корне проекта в терминале выполняем `"protoc --python_out=. proto/scan.proto"` и получаем файл `scan_pb2.py`, переносим его в папку с `python` кодом.

Код на сервере адаптивный к двум вариантам передачи: просто пакетами и `protobuf`-пакетами. Параметр `use_protobuf` отвечает за переключение режимов передачи (`True` - `protobuf`, `False` - `racket`). Запись логов происходит в файл `ble_log_csv` (файл перезаписывается при перезапуске сервера). Запись осуществляется в два разных файла в зависимости от режима передачи.

— Поточковая передача нескольких батчей (отправка по времени или по количеству).

Файлы `batch_builder.h` и `batch_builder.c`.

Параметры `MAX_BATCH_EVENTS` - максимальное количество `events` в сообщении,

`BATCH_TIMEOUT_MS` - максимальное количество времени между отправками.

В файле `scan.options` добавляем строку:

```
ripas.ScanBatch.ble_max_count:10
```

таким образом задается максимальное количество `BleEvent message` внутри `ScanBatch message`.

Перегенерируем `C`-код.

Перегенерируем `python` код.

Сделал правильный `decode` под `scanbatch` на сервере (файл `TCP_server`).

Буфер переделан под `scanbatch`: емкость буфера 128 сообщений (`BLE_BUFFER_CAPACITY 128`), максимальная величина сообщения 640 байт (`BLE_PACKET_MAX_SIZE 640`).

Примечание: при изменении величины сообщения в буфере `BLE_PACKET_MAX_SIZE`, следует также изменить величину `uint8_t buffer[640]` в `app.c`

и размер буфера в `pb_ostream_t stream = pb_ostream_from_buffer(buf + 2, 640)` в `batch_builder.c`.

Также после добавления `scan_batch` сильно увеличилась задержка между `utc esp` и `utc ntp`, поскольку сообщения теперь отправляются по времени или по количеству.

7. Системный лог (ошибки, перезапуски, сетевые события)

Цель: сделать логирование системных уведомлений.

Что нужно сделать:

— каждое системное уведомление отправляется по `UART` и сохраняется в отдельный файл;

— Поля: тип ошибки (`Info`, `Warn`, `Error`), `TAG` ошибки, описание, время `monotonic`, время `utc esp`.

Описания: переполнение буфера, установление/разрыв соединения `TCP`, синхронизация времени и т.д.

Готово, если:

— Файл создается;

— Логирование происходит корректно;

— Файл открывается для просмотра.

Сделано:

Все сделано.

8. Буферизация и очередь BLE-событий

Цель: реализовать буферизацию BLE-событий.

Что нужно сделать:

- реализовать очередь / ring buffer для ble_event_t;
- добавить mutex для защиты буфера от повреждений другими потоками.
- обработать переполнение;
- добавить packet builder;
- добавить логирование событий.

Готово, если:

- события не теряются при нагрузке;
- поведение при переполнении понятно;
- логика устойчива.

Сделано:

- Реализована очередь для событий.
 - Буфер для оффлайн режима реализован как кольцевой буфер (Ring Buffer).
 - При переполнении буфера старые строки перезаписываются новыми. При появлении подключения TCP, данные отправляются в порядке возрастания актуальности (старые данные отправляются первыми, а наиболее свежие - последними).
 - Mutex добавлен, критическая зона - место записи и чтения данных из буфера.
 - Сделан packet builder.
 - Python сервер тормозит. Решение - писать лог в CSV формате и потом конвертировать в Excel.
 - Сделал систему логирования данных в CSV и в Excel для анализа. Залогировал 100 тыс строк. Построил график задержки между реальным NTP (вычисленным по монотонному времени системы + смещение) и абсолютным временем ESP (вычисленным по тому же принципу). Получил рассинхронизацию времени от 8 мс до - 180 мс на дистанции в ~4 часа.
 - Отфильтровал все данные Медианным фильтром (с окном 71) с минимизацией скачков/импульсных выбросов и построил график. График имеет медленный (61 мс/ч) нисходящий тренд на протяжении всей работы, также видны сильные скачки раз в 60 минут, что соответствует интервалу синхронизации времени на ESP.
- Сделать синхронизацию времени сервера раз в 30 мин и использовать в качестве монотонного времени время работы программы, а не системное время. Сделать плавную корректировку offset на ESP, чтобы скорректировать боковой тренд графика.

9. Сборка и запуск проекта под Linux / WSL

Цель: поднять проект под Linux / WSL.

Что нужно сделать:

- установить ESP-IDF под Linux / WSL;
- собрать проект;
- выявить отличия от Windows;
- зафиксировать проблемы и решения.

Готово, если:

- проект собирается под Linux;
- есть инструкция запуска;
- известны различия сред.

Сделано:

Порядок запуска на linux

- Обновляем список пакетов: открываем терминал и выполняем "sudo apt update".
- Обновляем систему: выполняем "sudo apt upgrade -y".

Устанавливаем зависимости: "sudo apt install -y git wget flex bison gperf python3 python3-pip python3-venv cmake ninja-build ccache libffi-dev libssl-dev dfu-util libusb-1.0-0".

Устанавливаем ESP-IDF CLI, именно CLI (интерфейс командной строки) поскольку сама ESP-IDF весит 12 Гб, а CLI сильно меньше. Клонировать SDK: для этого переходим в домашний каталог "cd ~" и выполняем "git clone -b v5.5.1 --depth 1 https://github.com/espressif/esp-idf.git".

Устанавливаем только toolchain для ESP32: переходим в клонированный каталог "cd esp-idf" и выполняем "./install.sh esp32".

Активируем: выполняем "./export.sh". Это нужно делать каждый раз перед сборкой.

Теперь нужно создать общую папку для Windows и Linux. Для этого на основной системе (Windows) нужно создать папку в любом месте.

Далее запустить Oracle VirtualBox, выбрать "Ubuntu" и нажать "Настроить". В открывшемся окне -> Расширенные -> Общие -> Дополнительно -> Общий буфер обмена:

Двунаправленный, Функция Drag'n'Drop: Двунаправленный. Общие папки -> Добавить новую общую папку -> Указать путь до этой, ранее созданной, папки и поставить галочку "Авто-подключение". Нажать "Ок" и "Ок".

Запустить виртуальную машину и войти в систему. Слева вверху нажать "Устройства" и "Подключить образ Дополнений гостевой ОС". Появится образ "VBox_GAs_7.1.12".

Появится предупреждение - нажать "Запустить". Попросит ввести пароль - вводим и жмем "Подтвердить". Откроется терминал и в конце попросит перезапустить терминал - закрываем и снова открываем. Прописываем "sudo apt-get update", после "sudo apt-get install gcc make perl" и соглашаемся "Д". Закрываем терминал и перезагружаем систему. После перезагрузки добавляем пользователя в группу vboxsf. Открываем терминал и вводим "sudo usermod -a -G vboxsf user", вместо "user" вводим свое имя пользователя. Вводим пароль. И закрываем терминал.

Далее нужно извлечь образ "VBox_GAs_7.1.12".

Слева вверху нажать "Устройства" и "Подключить образ Дополнений гостевой ОС".

Появится образ "VBox_GAs_7.1.12". Появится предупреждение - нажать "Запустить".

Попросит ввести пароль - вводим и жмем "Подтвердить". Откроется терминал и в конце попросит перезапустить терминал - закрываем.

Снова извлекаем образ "VBox_GAs_7.1.12" и перезагружаем систему.

После всех действий в Файлах появится диск вида "sf__linux", в котором будут отображены файлы находящиеся на Windows.

На Windows: копируем папку с проектом в общую папку.

На Linux: копируем папку с проектом из общей папки в любую другую удобную директорию, для работы с ней непосредственно в Linux.

Проброс портов:

Заходим в Oracle VirtualBox, выбираем "Ubuntu" и нажимаем "Настроить". В открывшемся окне -> Расширенные -> USB -> Добавить новое USB-устройство. Смотрим и запоминаем какие устройства сейчас доступны. Подключаем USB программатор к ноутбуку и смотрим какое новое устройство появилось, его и добавляем (обычно FTDI USB <-> Serial [XXXX]). Жмем "Ок". Перезагружаем VirtualBox. Открываем терминал и вводим "ls /dev/ttyUSB*" для проверки, должно появиться "/dev/ttyUSB0".

Выдаем права на порт (получаем доступ к UART).

В терминале выполняем "sudo usermod -a -G dialout \$USER". Перезагружаем систему.

Сборка проекта и прошивка:

Сначала активируем окружение (ЭТО НУЖНО ДЕЛАТЬ КАЖДЫЙ РАЗ В НОВОЙ СЕССИИ ТЕРМИНАЛА, КОГДА НУЖНО СОБРАТЬ ПРОЕКТ). Переходим в каталог "cd esp-idf" и выполняем

"./export.sh".

Далее переходим в каталог проекта "cd

"/home/maksim/developments/RIPAS_work/Code/Espressif-IDE/wt32_eth01_ble_eth".

Проверяем что окружение активировалось, в папке с проектом выполняем "idf.py --version", должно вывести "ESP-IDF v5.5.1".

В папке с проектом выполняем "idf.py fullclean", а затем "idf.py build" для сборки проекта. В первый раз при сборке устанавливается большое количество дополнительных инструментов esp-idf, этот процесс занимает приличное количество времени ~40 минут, следует дождаться завершения процесса.

В случае если компилятор выдает какие-то ошибки и рекомендует выполнить "idf.py fullclean" - выполняем и следом вновь собираем проект.

Для прошивки контроллера выполняем "idf.py -p /dev/ttyUSB0 flash" предварительно переведя контроллер в режим bootload (как переводить в этот режим описано в другой инструкции). Для просмотра данных, приходящих по UART, выполняем "idf.py -p /dev/ttyUSB0 monitor".

Важно: в путях не должно быть пробелов, скобок и специальных символов, иначе терминал будет ругаться.

Запуск python-сервера:

Переходим в каталог с python кодом "cd

/home/maksim/developments/RIPAS_work/Code/python". Запускаем сервер "python3 TCP_server.py". При первом запуске попросит установить необходимые библиотеки - устанавливаем. Для установки в том же каталоге используем команды типа:

"pip install protobuf grpcio-tools",

"pip install ntplib".

Также если появляется ошибка "ModuleNotFoundError: No module named 'google'", следует установить protobuf. В терминале выполнить "pip3 install protobuf".

Может быть такое, что protobuf установлен, но python его не видит, тогда в терминале выполняем "which python3" и "which pip3", они должны быть из одной среды.

Также требуется создать виртуальное окружение, для этого "cd ~" и "cd

/home/maksim/developments/RIPAS_work/Code/python

python3 -m venv venv" и активировать его "source venv/bin/activate". Активировать виртуальное окружение нужно каждый раз при запуске терминала.

После этого установить protobuf "pip install protobuf". Запустить сервер "python3 TCP_server.py". Возможно снова потребуются доустановить библиотеки.

После в терминале начнется выполнение программы.

Подключаем LAN кабель к компьютеру и включаем контроллер.

Настройка Ethernet в VirtualBox:

Запустить Oracle VirtualBox, выбрать "Ubuntu" и нажать "Настроить". В открывшемся окне - > Расширенные -> Сеть.

Для "Адаптер 1" устанавливаем галочку "Включить сетевой адаптер", Тип подключения: Сетевой мост, Тип адаптера: Intel PRO/1000 MT Desktop (82540EM), устанавливаем галочку "Подключить кабель".

Для "Адаптер 2" устанавливаем галочку "Включить сетевой адаптер", Тип подключения: NAT, Тип адаптера: Intel PRO/1000 MT Desktop (82540EM), устанавливаем галочку "Подключить кабель".

Запускаем виртуальную машину. В терминале выполнить "ip a". Увидим что для enp0s3 и для enx00e04c460448 (это esp) не назначен IP.

Открываем настройки сети в linux. Для Ethernet подключения открываем "Параметры сети" и во вкладке IPv4 выбираем метод "Вручную", задаем ip адрес "192.168.1.2" и маску "255.255.255.0", нажимаем применить.

Задаем вручную: в терминале выполняем "sudo ip addr add 192.168.1.2/24 dev enp0s3" и "sudo ip link set enp0s3 up". Теперь при выполнении "ip a" увидим что enp0s3 получил свой адрес.

Или открываем настройки сети в linux. Для enp0s3 подключения открываем "Параметры сети" и во вкладке IPv4 выбираем метод "Вручную", задаем ip адрес "192.168.1.2" и маску "255.255.255.0", нажимаем применить.

Для wt32-eth01 нужно тоже настроить IP. Открываем файл eth.c в папке main проекта.

Ищем и раскомментируем эти строки:

```
//ESP_ERROR_CHECK(esp_netif_dhpcp_stop(netif));  
// Назначаем статический IP  
//esp_netif_ip_info_t ip_info;  
//IP4_ADDR(&ip_info.ip, 192, 168, 1, 10);  
//IP4_ADDR(&ip_info.gw, 192, 168, 1, 1);  
//IP4_ADDR(&ip_info.netmask, 255, 255, 255, 0);  
//ESP_ERROR_CHECK(esp_netif_set_ip_info(netif, &ip_info));
```

Сохраняем файл.

Открываем файл tcp_client.c в папке проекта. Ищем эти строки:

```
//#define SERVER_IP "192.168.1.1" // Для Linux  
#define SERVER_IP "192.168.137.1" // Для Windows  
#define SERVER_PORT 9000
```

Закомментировать строчку "Для Windows" и раскомментировать "Для Linux". Также "для linux" установить значение "192.168.1.2" (это тот же адрес, который мы задавали в настройках сети Linux).

Сохраняем файл, собираем проект и прошиваем контроллер.

Убрать нахуй ебанный DHCP в инструментах виртуалбокс.

В Oracle VirtualBox заходим в "Файл" -> "Инструменты" -> "Менеджер сетей", выбираем "VirtualBox Host-Only Ethernet Adapter", внизу на вкладке "Адаптер" ставим галочку "Настроить адаптер вручную" и устанавливаем IP адрес 192.168.56.1, маску сети 255.255.255.0. Переходим внизу на вкладку "DHCP сервер" и убираем галочку с "Включить сервер".

Вводим в терминале "sudo ip addr add 192.168.56.2/24 dev enx00e04c460448" и "sudo ip link set enx00e04c460448 up". Затем "sudo ip addr add 192.168.56.101/24 dev enp0s3".

Открываем файл eth.c в папке main проекта. И делаем:

```
IP4_ADDR(&ip_info.ip, 192, 168, 56, 10);  
IP4_ADDR(&ip_info.gw, 192, 168, 56, 2);  
IP4_ADDR(&ip_info.netmask, 255, 255, 255, 0);
```

Открываем файл tcp_client.c в папке проекта. И делаем:

```
#define SERVER_IP "192.168.56.2" // Для Linux
```

Сохраняем и прошиваем.

Выполняем в терминале "sudo sysctl -w net.ipv4.ip_forward=1" и "sudo ip route add 192.168.56.0/24 dev enx00e04c460448"

Короче все это хуйня и нужно делать Host-only